«Справедливое» разделение канала

На просторах хабра мне попалась пара статей «Mikrotik Router OS, скрипт для динамического деления скорости». А поскольку этой проблемой я занимаюсь уже не первый год, я решил поделиться своими знаниями.

Прежде всего, эта статья пригодится тем, кто имеет желание «справедливо делить интернет» между несколькими клиентами сети. Однако решение подойдет и для разделения канала связи нескольких офисов или доступа к отдельным ресурсам, и для шейпинга.

Введение

Под «справедливым» разделением канала я подразумеваю утилизацию всей его пропускной способности независимо от количества потребителей. А потребители должны получать канал поровну.

Поскольку я работаю из-под windows, то для удобства я буду использовать WinBox, позволяющий конфигурировать маршрутизатор при помощи визуальных оснасток. С моей точки зрения, использовать WinBox гораздо удобнее, чем командную строку т.к. при наличии множества правил, найти необходимые гораздо проще. Да и скорость навигации значительно увеличивается.

Пользователям Linx- и Unix-систем не стоит расстраиваться. WinBox отлично работает под вайном. Кроме того, MikroTik обладает удобным веб конфигуратором WebFig.

Далее в статье я буду приводить текстовые описания своих действий и пояснять их графикой. В конце статьи я приведу ссылку на файл конфигурации раутера, чтобы вы могли изучить его, залить в свой раутер и проверить. Для этих целей я рекомендую использовать версии MiktoTik для x86 совместимых систем и виртуальные машины.

Также в конце статьи я постараюсь привести все полезные ссылки. Если чего-то не найдете – пожалуйста, сообщите мне об этом и я постараюсь дополнить статью.

Постановка задачи

Сначала я опишу некоторые задачи, которые вы сможете решить, изучив эту статью. Заранее сделаю две оговорки:

1. все описанные задачи должны выполняться при любом количестве потоков загрузки/отдачи каждым потребителем.
2. Под каналом я подразумеваю и входящий, и исходящий каналы.

Требуется:

1. Справедливо разделить входящий и исходящий каналы между пользователями:
   - При полной утилизации пропускной способности канала всеми потребителями, канал должен быть разделен между ними поровну.
   - При свободном канале или малой утилизации пропускной способности канала одними потребителями, другим потребителям должна быть предоставлена вся оставшаяся пропускная способность канала, но не более.
2. Гибко разделить входящий и исходящий каналы между пользователями:
   - Должна быть возможность ограничить ширину входящего/исходящего канала любого пользователя или до любого внешнего ресурса.
   - Должна быть возможность необходимым сервисам предоставлять требуемую для их нормального функционирования ширину канала/скорость.
3. Сделать быстрой загрузку Web страничек независимо от текущей утилизации канала.

Немного разъяснений.

Пункт первый.

Например, пропускная способность входящего канала составляет 1000 кб/с.
Если, например, Main_HomePC получает данные из интернета со скоростью 100 кб/с, тогда Second_HomePC должен иметь возможность получать данные со скоростью до 900 кб/с.

При увеличении скорости отдачи данных удаленным сервером для Main_HomePC до, скажем, 800 кб/с, канал должен быть разделен между потребителями поровну (т.е. по 500 кб/).
Для решения этой задачи служит тип очереди PCQ (но об этом ниже).

Пункт второй.

Если в сети есть такие потребители, которым нужен доступ в интернет, но вы не хотите, чтобы они «откусывали» большой кусок канала, им следует ограничить скорость, либо сделать так, чтобы канал между ними и остальными потребителями разделялся не поровну.
Для решения такой задачи служит тип организации очередей Queue Tree (теория уже близко).

При этом можно, например, выделить больший приоритет соединениям к определенным серверам. Например, я так добивался великолепной работы RDP соединений вне зависимости от загрузки канала. 1С суппорты были очень довольны.

Пункт третий.

Все же, серфинг – это самое заметное занятие человека, использующего интернет. Если пользователь будет вынужден долго ждать открытия страничек, то никакие очереди и разделения канала ему не помогут остаться довольным. Посему постараемся ускорить открытие страничек независимо от пользователя и текущей загрузки канала.

Итак, мы пришли к тому, что использовать нужно организацию очередей Queue Tree и тип очереди PCQ.
Прежде, чем решить поставленные задачи, обратимся к теории.

Теория

Я буду стараться избавляться от копи-паста. Поэтому буду приводить ссылки на теорию, а самое важное буду описывать кратко. Полная теория понадобятся тем, кто захочет разобраться в происходящем более подробно. Если же вы хотите настроить раутер по приведенному мной примеру, то можете обратиться в самый конец статьи, где найдете ссылки на конфигурационные файлы.

Сначала ознакомьтесь с тем, как трафик проходит через роутер - http://wiki.mikrotik.com/wiki/Manual:Packet_Flow
Достаточно глазами изучить первую диаграмму и принцип станет понятен.

HTB

Об HTB можно почитать здесь - http://wiki.mikrotik.com/wiki/Manual:HTB

Вкратце, HTB (Hierarchical Token Bucket) – классический метод построения очередей для обработки различного вида трафика.

Метод основан на трех простых, но обязательных этапах:

1. Выделение и маркировка интересующего нас трафика.
2. Создание правил для применения к отдельным типам трафика отдельных очередей.
3. Применение выбранных стратегий (правил и дополнительных параметров) к интерфейсам.

Реализуется HTB только через Queue Tree.

Узнать, в чем отличие такого типа очередей от Simple Queue вы можете здесь - http://wiki.mikrotik.com/wiki/Manual:Queue#Queue_Tree

Что касается типа очередей, то был выбран PCQ.

Дело в том, что к каждому интерфейсу маршрутизатора применяется очередь для отправки пакетов. Пакеты собираются в буфер, а затем вытаскиваются из него и последовательно отправляются на физический сетевой порт. Порядок вытаскивания пакетов из буфера называется очередью.

Пакеты, вытаскиваемые из буфера сетевого интерфейса, классифицируются раутером по некоторым признакам и помещаются в, так называемые, «потоки» пакетов. Разные типы очередей по-разному классифицируют пакеты и помещают их в потоки, используя разные принципы.

Изначально для всех интерфейсов применяется очередь типа PFIFO (packet first in first out, первый вошел – первый вышел). Такая очередь не позволит хоть как-то контролировать порядок отправки пакетов. Все пакеты помещаются раутером в один поток и поэтому, при таком типе очереди, мы не можем решить ни задачу 1, ни задачу 2 полностью. Можем лишь ограничить скорость, но не можем балансировать нагрузку. Задача 3 решается т.к. здесь нам не нужна балансировка.

Нам нужно настроить раутер так, чтобы пакеты от разных пользователей помещались в разные потоки. Тогда нам будет не важно, сколько соединений установил пользователь.

Тип очереди PCQ (Per Connection Queuing, помещение в очередь по соединениям) позволяет выбрать классификаторами разделения на потоки адреса и порты источника и назначения пакетов, что позволит применять правила к отдельным потокам, а не к отдельным соединениям или пакетам. Этот тип очереди позволяет ограничивать скорость выборки пакетов из буфера и помещать их в разные потоки (это один из вариантов выделения фиксированного канала), а в случае установления значения Rate=0, все потоки, выделяемые этой очередью, будут получать одинаковую ширину канала (вариант динамического разделения канала; так будет работать разделение интернета между Wireless Users).

Более подробно можно прочесть здесь - http://wiki.mikrotik.com/wiki/Manual:Queue

Mangle

Первый этап HTB.

Чтобы применить правила очередности прохождения пакетов через раутер, пакеты нужно маркировать.

При прохождении через раутер, пакет может быть промаркирован, а затем быстро найден по маркеру.

Это нужно для того, чтобы применять общие правила к разным пакетам, или разные правила для схожих пакетов.

Много текста здесь излишне. Более подробно вы можете почитать здесь - http://wiki.mikrotik.com/wiki/Manual:IP/Firewall/Mangle

Queue Tree

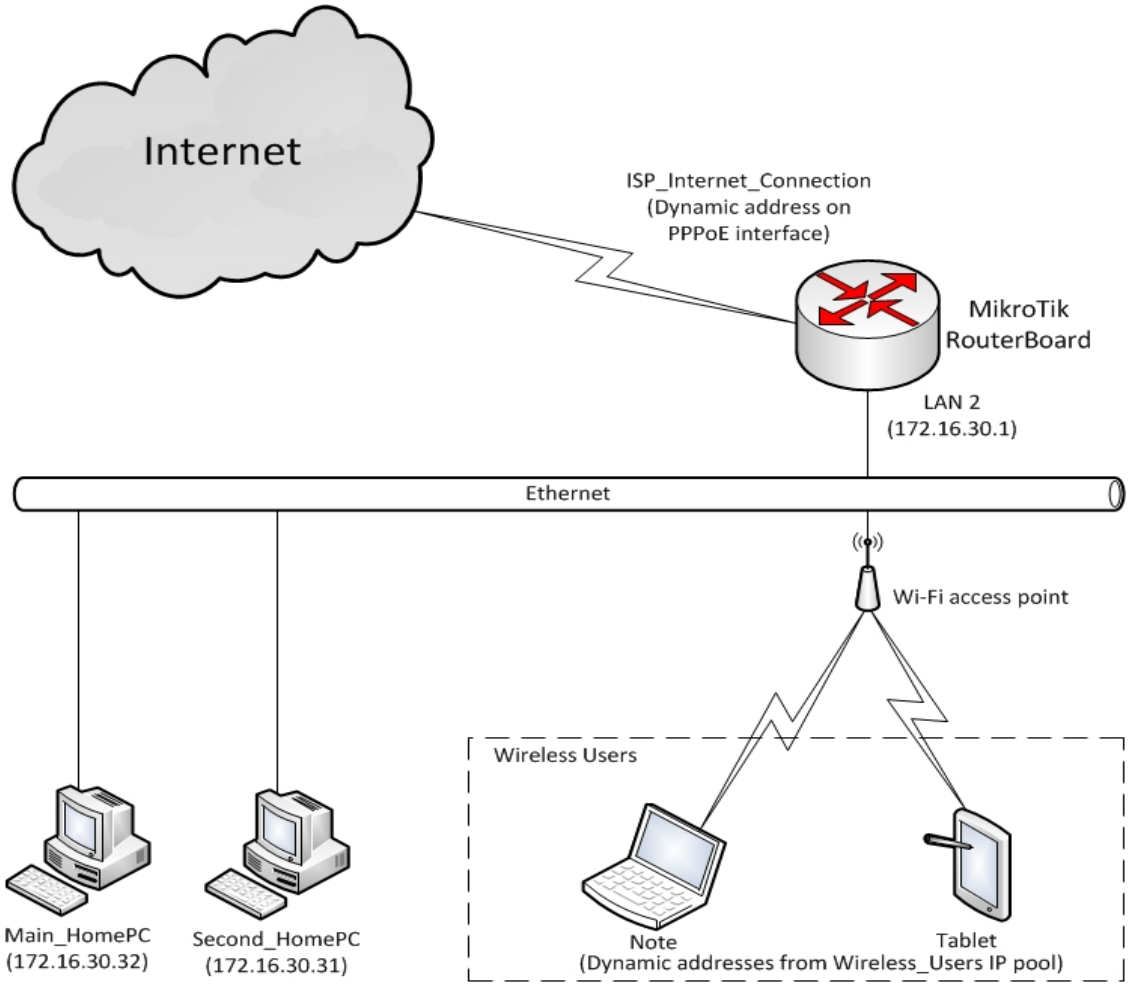
Второй этап HTB – создание правил, позволяющих к разным типам трафика применять разные типы очередей.

Написать здесь что-то своими словами достаточно сложно, а от копипаста я отказался. Поэтому либо читаем теорию, либо смотрим практику. Возможно, проще будет сначала увидеть реализацию.

Практика

Топология сети

Моя домашняя сеть имеет следующую топологию:



Маршрутизатор подключается к PPPOE серверу провайдера интерфейсом PPPoE_Client, названным мною «ISP_Internet_Connection» и через этот интерфейс получает доступ к сети интернет. При этом параметры сети (IP, Gate, DNS) выдаются динамически.

Main_HomePC и Second_HomePC – настольные компьютеры. Они имеют статические адреса, на примере которых я покажу, как можно сбалансировать нагрузку на канал между ними.

Note и Tablet – примеры устройств, использующих доступ через Wi-Fi (будут динамически включаться в группу wireless_users, адреса им будут выдаваться из пула wireless_users) и, как правило, требующих получения адреса от DHCP сервера т.к. могут быть использованы в разных сетях (посему, зачастую, не должны иметь жестко вбитых настроек). У меня есть только смартфон, но для полноты примера я вбил и notebook. Вообще количество таких устройств будет зависеть лишь от выделенного им пула адресов.

Mangle

Первое, что необходимо сделать – промаркировать пакеты от всех устройств-потребителей.

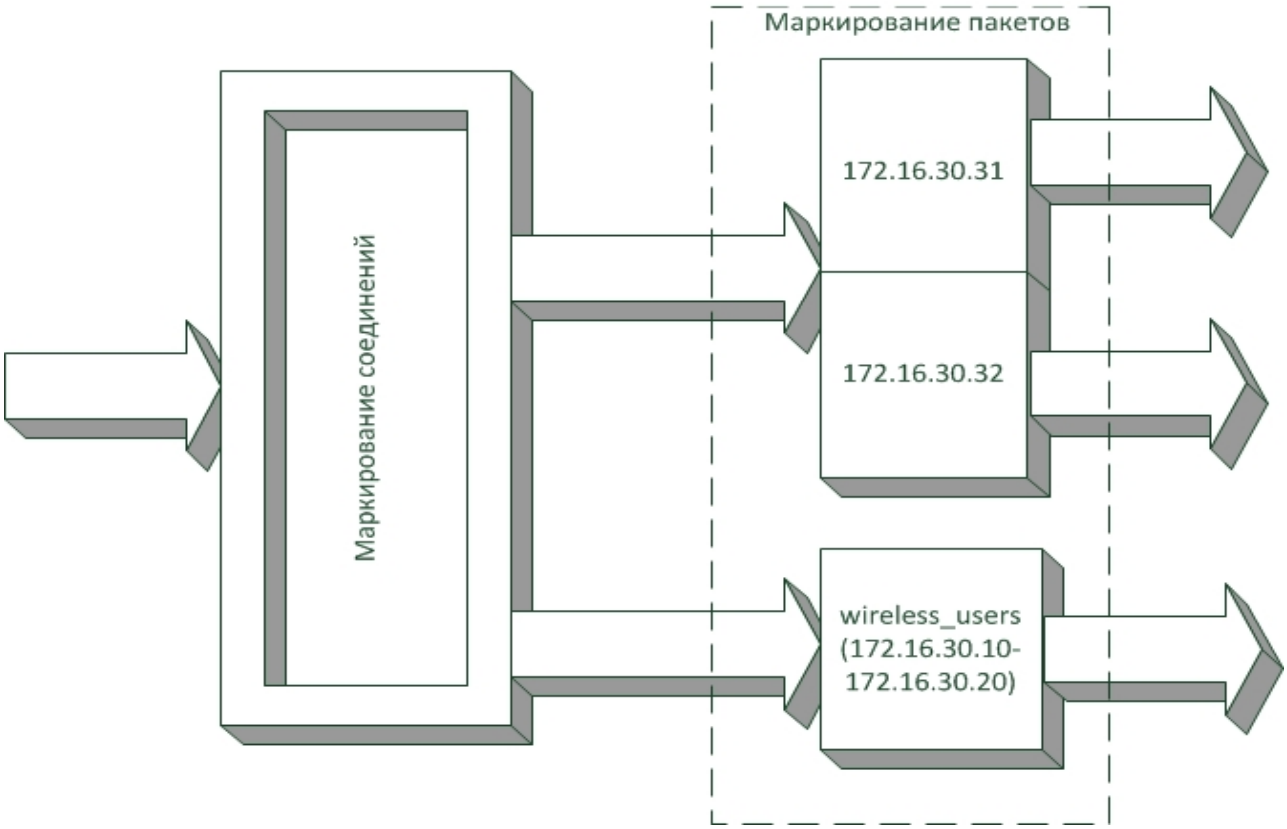
Мы сначала будем маркировать соединения, а уж потом – пакеты, относящиеся к соединениям. В таком случае раутер не перепроверяет содержимое заголовков пакетов, принадлежащих уже промаркированным соединениям. Такой подход позволит не допустить излишнюю загрузку процессора раутера.

Если вы уверены, что раутер справится и с маркированием каждого пакета – соединения можно не маркировать.

Настройки маркировщика пакетов можно найти в разделе Ip->Firewall->Mangle.

Маркирование будем проводить в ветви forward. Здесь раутеру уже известно, кому пакет будет отправлен на выходе, а так же применены правила firewall, а значит — не будут маркироваться лишние пакеты.

Делать это будем по такому принципу:



Таким образом мы разделим пакеты отдельно к каждому из домашних компьютеров и отдельно для группы беспроводных устройств.

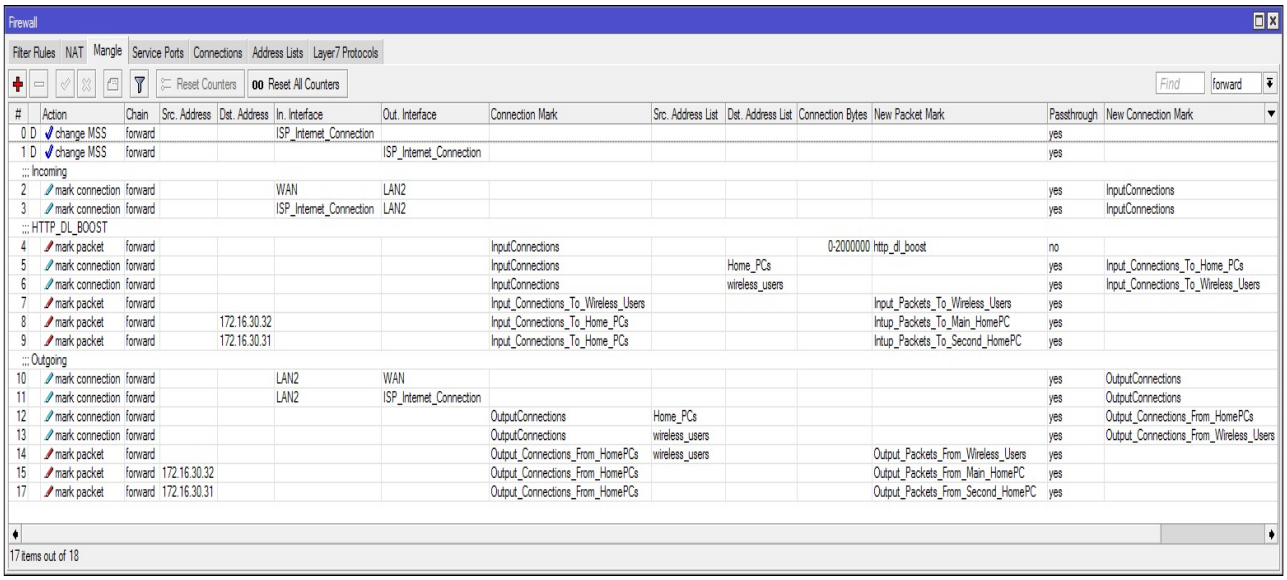
Такой алгоритм нужно применить к пакетам, проходящим от LAN к WAN и от WAN к LAN.

1. Маркируем соединения, устанавливаемые из внешней сети (я одним маркером помечаю соединения с двух интерфейсов, но в классическом случае для доступа к интернет используется только один интерфейс и поэтому правила маркировки соединения от и для WAN можно опустить).

Для этого маркируем все соединения, входящие на WAN и выходящие из LAN. Для того, чтобы разделить пакеты для беспроводных устройств и домашних компьютеров, я применил правила маркировки только для выбранных групп адресов.

2. Маркируем пакеты внутри промаркированных соединений. Отдельно я маркирую пакеты с адресом назначения 172.16.30.31, отдельно 172.16.30.32 и отдельным маркером помечаю пакеты для пула 172.16.30.10-172.16.30.20.

Аналогичные правила создаем для исходящих пакетов.
Итог выглядит так:



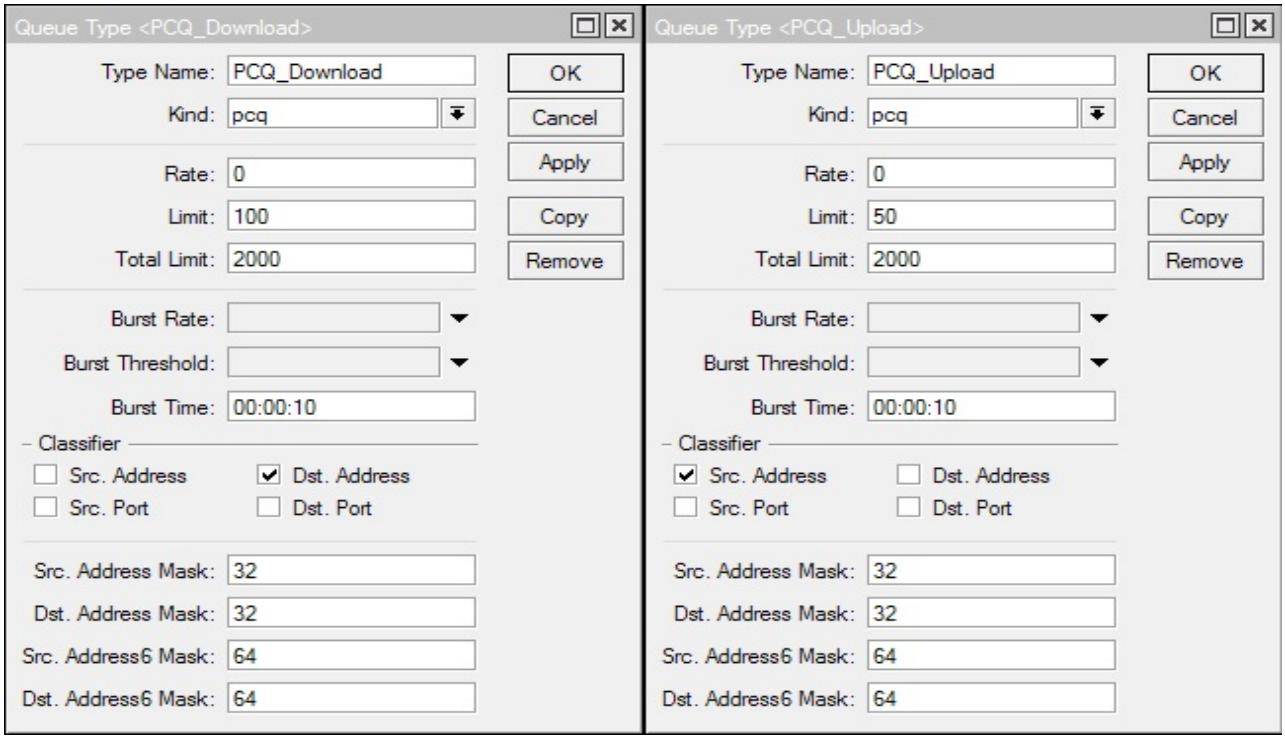
Правила 0 и 1 помечены литерой D, что означает dynamic. Эти правила создаются автоматически при установлении PPPoE соединения. Так может быть не везде. Они не относятся к теме статьи.

Правило 4 не полностью отражено на рисунке. Дополнительно следует указать протокол IP и порт-источник 80. Правило прерывает цепочку для того, чтобы под маркером http_dl_boost были лишь первые два миллиона байт каждого нового соединения (думаю, этого достаточно для загрузки большинства страниц).

По такому принципу возможно построить и upload booster, а так же заставлять быстрее проходить мелкие пачки данных, например при игре в heroes3 на отдаче хода можно ускорить передачу данных.

Queue

Второй шаг – настройка очередей. Для начала создадим два новых типа очередей для входящего и исходящего трафика. Тип PCQ.



Потоки пакетов, к которым будет применен тип очереди PCQ_Download, будут классифицироваться по адресу назначения и каждому из них будет присвоен одинаковый приоритет. Такая очередь обеспечит равноправие среди потребителей независимо от количества потоков на каждого из них.

Аналогично очередь для исходящих пакетов будет классифицировать потоки по адресу отправителя.

На основе этого типа очередей возможно так же уравноправить потоки пакетов для разных приложений одного пользователя (классификаторы: адрес и порт назначения). Сложность может возникнуть только с торрентами. Но при должной усидчивости и это можно решить, применив сложное дерево очередей.

Если вы заметили, параметр Limit различен для входящих и исходящих пакетов. Чтобы понять, для чего это нужно, вернитесь к теории об HTB.

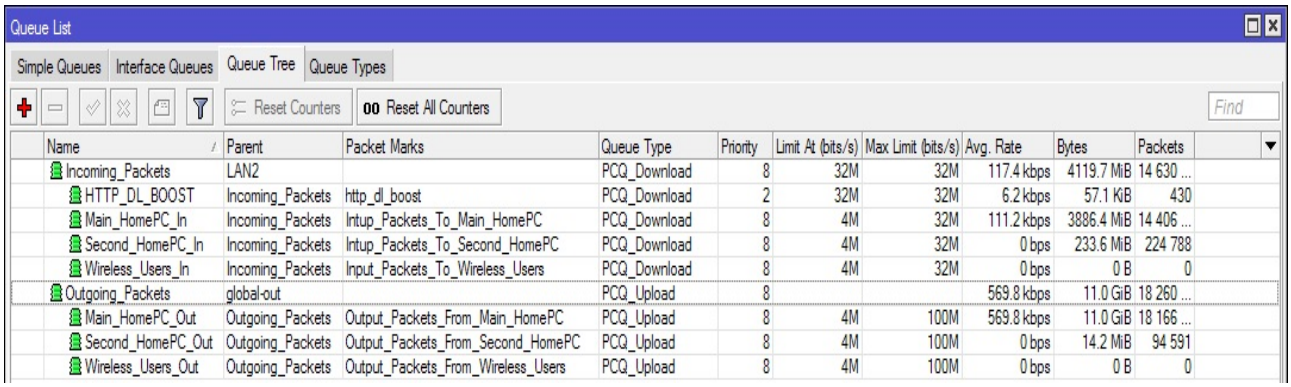
Теперь все готово для создания своего дерева очередей.

Нас интересуют две ветви:

— входящие пакеты (так я назову пакеты, которые приходят из интернета и предназначены устройствам в локальной сети).

— исходящие пакеты (пакеты, исходящие от устройств в локальной сети).

Вот так выглядят все правила дерева очередей.



Name	Parent	Packet Marks	Queue Type	Priority	Limit At (bits/s)	Max Limit (bits/s)	Avg. Rate	Bytes	Packets
Incoming_Packets	LAN2		PCQ_Download	8	32M	32M	117.4 kbps	4119.7 MiB	14 630 ...
HTTP_DL_BOOST	Incoming_Packets	http_dl_boost	PCQ_Download	2	32M	32M	6.2 kbps	57.1 KiB	430
Main_HomePC_In	Incoming_Packets	Intup_Packets_To_Main_HomePC	PCQ_Download	8	4M	32M	111.2 kbps	3886.4 MiB	14 406 ...
Second_HomePC_In	Incoming_Packets	Intup_Packets_To_Second_HomePC	PCQ_Download	8	4M	32M	0 bps	233.6 MiB	224 788
Wireless_Users_In	Incoming_Packets	Input_Packets_To_Wireless_Users	PCQ_Download	8	4M	32M	0 bps	0 B	0
Outgoing_Packets	global-out		PCQ_Upload	8			569.8 kbps	11.0 GiB	18 260 ...
Main_HomePC_Out	Outgoing_Packets	Output_Packets_From_Main_HomePC	PCQ_Upload	8	4M	100M	569.8 kbps	11.0 GiB	18 166 ...
Second_HomePC_Out	Outgoing_Packets	Output_Packets_From_Second_HomePC	PCQ_Upload	8	4M	100M	0 bps	14.2 MiB	94 591
Wireless_Users_Out	Outgoing_Packets	Output_Packets_From_Wireless_Users	PCQ_Upload	8	4M	100M	0 bps	0 B	0

Родитель ветви Incoming_Packets – интерфейс LAN2 – с него пакеты отправляются на локальные устройства.

Родитель ветви Outgoing_Packets – global-out т.к. пакеты в моем случае могут быть отправлены и с WAN, и с ISP_Internet_Connection.

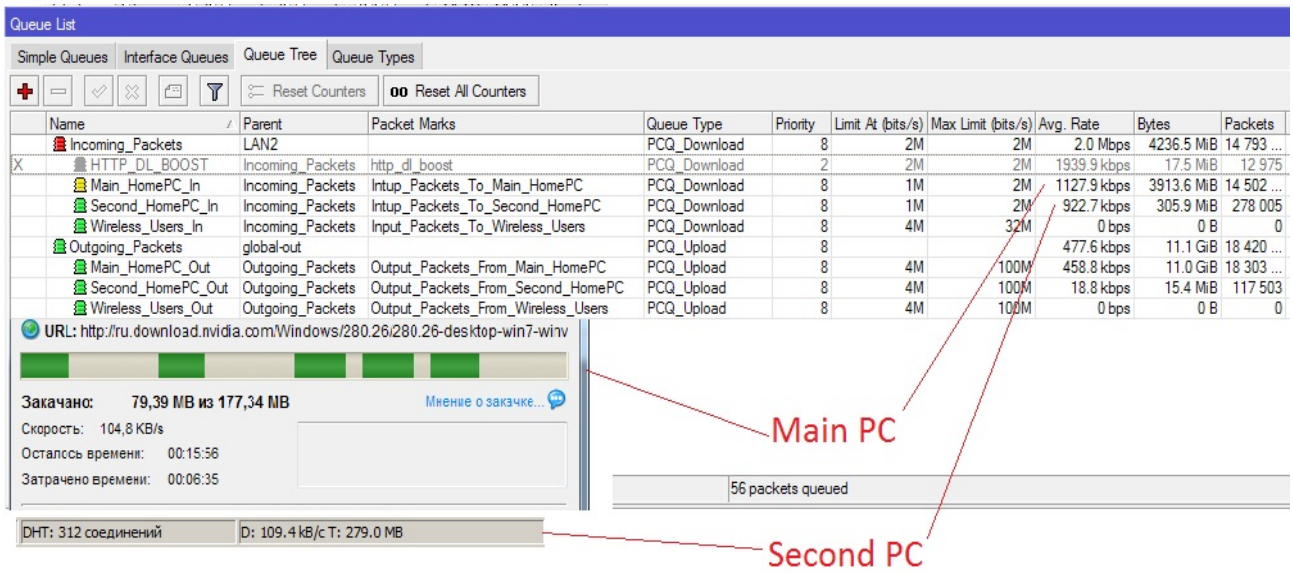
Обратите внимание, приоритет HTTP_DL_BOOST равен 2, а так же LimitAt=MaxLimit=32M. Это дает высокий приоритет и максимальную полосу первым ~2Мб трафика, получаемого с 80 порта по IP, что обеспечит быстрое открытие страничек даже на медленных каналах (проверено на 2 Мбит/с канале при трех пользователях).

Результаты.

Чтобы провести полноценный эксперимент, я на MainPC запускаю закачку в 5 потоков с nvidia.com, а на SecondPC запускаю закачку торрентом.

Поскольку забить все 32 мегабита почти не реально, я искусственно ограничу полосу до 2 мегабит.

Для чистоты эксперимента я отключил DL_BOOST.



Цифры немного не совпадают т.к. скриншоты делались не в одно мгновение.

Отключенное правило не обновляется и поэтому цифры в нем не соответствуют моменту.

P.S. Нужно добавить, что в моем случае (Локальные ресурсы провайдера находятся за WAN интерфейсом и доступ к ним я получаю на всех 32 Мбит/с, в то время как доступ к интернет у меня 8/8 Мбит/с) нужно подойти немного иначе и потоки для внутренних ресурсов выделить в отдельные ветви. Тогда можно будет разделять канал еще гибче. Но внутренние ресурсы я не использую, поэтому привел все в более простом виде.